

TypeScript

Crash Course

Acknowledgement

- These slides are based on The Coincise TypeScript Book by Simone Poggiali: <https://github.com/gibbok/typescript-book/>
- Fatemeh Qasemkhnai “TypeScript – Zero maybe not to hero”

What is TypeScript?

- TypeScript is a strongly typed superset of JavaScript that adds optional static typing
- Created by Anders Hejlsberg in 2012 and maintained by Microsoft as an open-source project
- TypeScript is neither an interpreted nor a compiled language
- TypeScript is *transpiled* to JavaScript, making it executable in any JavaScript runtime (e.g., browser, Node.js)
- TypeScript supports multiple programming paradigms: functional, generic, imperative, object-oriented

Why TypeScript?

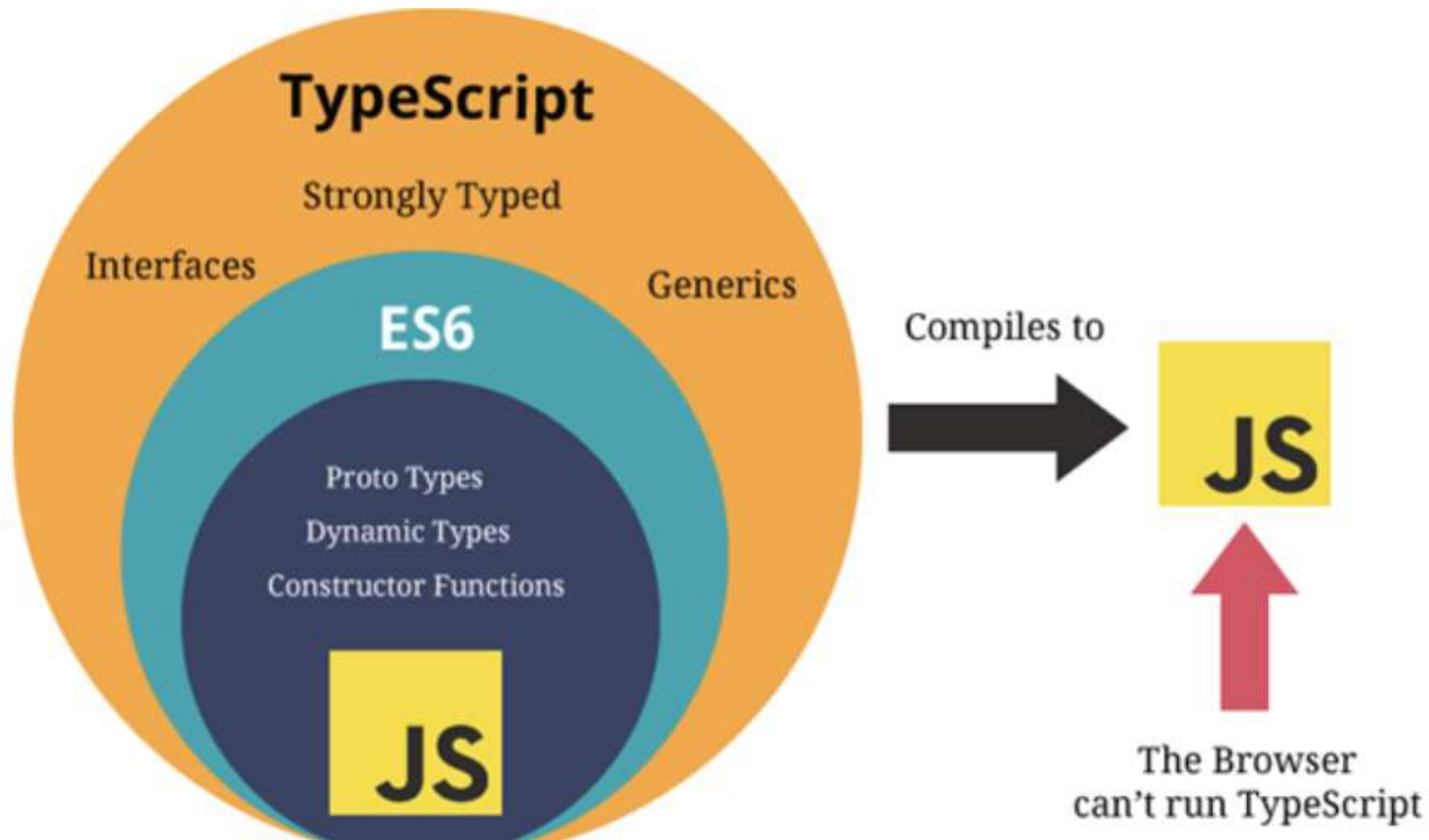
- Being strongly typed, TypeScript helps prevent programming mistakes before execution, thanks to static type checking
- It enables developers to express constraints and behaviors on data, improving correctness and maintainability
- Especially valuable in large-scale or long-term projects
- TypeScript also provides:
 - Type inference to reduce boilerplate
 - Access to modern JavaScript features (ES6/ES7+)
 - Cross-platform and cross-browser compatibility
 - Excellent tooling support, including IntelliSense and powerful refactoring

Polyfilling and Transpilation

- TypeScript does not run natively in any environment, it must be transpiled to JavaScript to run in the browser
- Transpilation is the process of converting TS syntax to older, compatible JS
- TypeScript uses tools like `tsc` (TypeScript compiler) to generate JavaScript
- Polyfilling, the technique to provide missing features expected by modern JS, can be used to fill compatibility gaps in older environments

```
// TypeScript
const sum = (a: number, b: number): number => a + b;
// Transpiled to JavaScript
const sum = (a, b) => a + b;
```

TypeScript and JavaScript



TypeScript and JavaScript

- TypeScript is written in .ts or .tsx files, while JavaScript is written in .js or .jsx files.
- Files with the extension .tsx or .jsx can contain JavaScript Syntax Extension JSX, which is used in React for UI development.
- TypeScript is a typed superset of JavaScript (ECMAScript 2015) in terms of syntax.
- All JavaScript code is valid TypeScript code, but the reverse is not always true.

TypeScript and JavaScript

- For instance, consider a function in a JavaScript file with the `.js` extension, such as the following:

```
const sum = (a, b) => a + b;
```

- The function can be converted and used in TypeScript by changing the file extension to .ts.
- However, if the same function is annotated with TypeScript types, it cannot be executed in any JavaScript runtime without compilation.
- The following TypeScript code will produce a syntax error if it is not compiled:

```
const sum = (a: number, b: number): number => a + b;
```


TypeScript and JavaScript

- TypeScript was designed to detect possible exceptions that can occur at runtime during compilation time by having the developer define the intent with type annotations.
- In addition, TypeScript can also catch issues if no type annotation is provided.
- For instance, the following code snippet does not specify any TypeScript types:

```
const items = [{ x: 1 }, { x: 2 }];  
const result = items.filter(item => item.y);
```

- In this case, TypeScript detects an error and reports:

Property 'y' does not exist on type '{ x: number; }'

TypeScript and JavaScript

- The runtime behaviour of JavaScript primarily influences TypeScript's type system.
- For example, the addition operator (+), which in JavaScript can either perform string concatenation or numeric addition, is modelled in the same way in TypeScript:

```
const result = '1' + 1;  
// Result is of type string
```

TypeScript and JavaScript

- The team behind TypeScript has deliberately decided to flag unusual usage of JavaScript as an error.
- For instance, consider the following valid JavaScript code:

```
const result = 1 + true; // In JavaScript, the  
                           result is equal 2
```

- TypeScript throws an error:

Operator '+' cannot be applied to types 'number' and 'boolean'.

Installation

- It is recommended that TypeScript be installed project-wise

```
npm install typescript
```

- rather than globally, as it provides a more predictable build process.

```
npm install -g typescript
```

Installation

During the TypeScript installation, two executables are installed: "tsc" as the TypeScript compiler and "tsserver" as the TypeScript standalone server.

The standalone server contains the compiler and language services that editors and IDEs can utilize to provide intelligent code completion.

Additionally, several TypeScript-compatible transpilers are available, such as Babel (via a plugin) or swc.

These transpilers can convert TypeScript code into other target languages or versions.

Transpiling the code

- TypeScript can be configured using the tsc CLI options or by utilizing a dedicated configuration file called tsconfig.json placed in the project's root.
- To generate a tsconfig.json file prepopulated with recommended settings, you can use the following command:

```
tsc --init
```

- When executing the `tsc` command locally, TypeScript will compile the code using the configuration specified in the nearest tsconfig.json file.

Transpiling the code

- Here are some examples of CLI commands that run with the default settings:

```
tsc main.ts // Compile a specific file (main.ts) to JavaScript
```

```
tsc src/*.ts // Compile any .ts files under the 'src' folder to JavaScript
```

```
tsc app.ts util.ts --outfile index.js // Compile two TypeScript files (app.ts and util.ts) into a single JavaScript file (index.js)
```

Structural Typing

- TypeScript is based on a structural type system
- In a structural system (like Python or JavaScript), type compatibility depends on the shape of the data, not on the name or declaration site
- This contrasts with nominative systems (like C# or Java), where compatibility is based on explicit declarations
- TypeScript's design reflects JavaScript's runtime duck typing

Primitive Types

- TypeScript supports seven primitive types.
- A primitive data type refers to a type that is not an object and has no associated methods.
- In TypeScript, all primitive types are immutable, meaning their values cannot be changed once assigned.
 - string – textual data (e.g. "hello")
 - number – integers and floats (e.g. 42, 3.14)
 - boolean – true or false values (e.g. true)
 - null – intentional absence of any value
 - undefined – uninitialized value
 - symbol – unique, immutable identifiers
 - bigint – large integers beyond Number.MAX_SAFE_INTEGER
- Use lowercase type names (string, number, etc.).
- Avoid String, Number, Boolean (they refer to object wrappers).

Primitive Types – number, string, boolean

```
let first: number = 123; // decimal -> 123
let second: number = 0x37CF; // hexadecimal -> 14287
let third: number = 0o377; // octal -> 255
let fourth: number = 0b111001; // binar -> 57
```

```
let employeeName: string = 'John Smith';
let employeeName2: string = "John Smith";
```

```
let isPresent: boolean = true;
```

Duck Typing

- The following example is a valid TypeScript code.
- As you can observe, "X" and "Y" have the same member "a," even though they have different declaration names.
 - Duck Typing: *"If it walks like a duck and quacks like a duck, then it probably is a duck."*
- TypeScript can infer types when no annotation is provided during:
 - Variable initialization.
 - Member initialization.
 - Setting defaults for parameters.
 - Function return type.

```
1  type X = {  
2      a: string;  
3  };  
4  type Y = {  
5      a: string;  
6  };  
7  const x: X = { a: 'a' };  
8  const y: Y = x; // Valid  
9  |
```

Type Inference

Example:

```
let x : string = 'x'; // Explicit typing  
let y = 'y'; // The type inferred is string
```

The TypeScript compiler analyzes the value or expression and determines its type based on the available information.

Return Values

- Functions can be defined using the following:

```
function add(a: number, b: number): number {  
    return a + b;  
}
```

- The following expression is equivalent to the previous, the compiler automatically infers the type of the return value is a `number`

```
function add(a: number, b: number) {  
    return a + b;  
}
```

- The special type `void` is available for function return parameters.

Literal Types

- A Literal Type is a single element set from a collective type, it defines a very exact value that is a JavaScript primitive.
 - Literal Types: number, string, boolean
 - Represent precise values at compile time
 - Do not exist at runtime – they are used only by the TypeScript compiler
- Example of literals:
 - `const a = 'a'; // String literal type`
 - `const b = 1; // Numeric literal type`
 - `const c = true; // Boolean literal type`

Literal Types

- String, Numeric, and Boolean Literal Types are used in the union, type guard, and type aliases.
- In the following example, you can see a type alias union, O can be the only value specified and not any other string:

```
type O = 'a' | 'b' | 'c';
```

Enum

- In TypeScript, an `enum` is a set of named constant values.
- Unlike literal types, enums exist at runtime.
- They are compiled into actual JavaScript objects:

```
enum Color {  
    Red = '#ff0000',  
    Green = '#00ff00',  
    Blue = '#0000ff',  
}
```

- This means enums can be used:
 - in logs
 - in loops
 - in comparisons at runtime

Generics

- Generics allow you to create reusable components and functions that can work with multiple types.
- With generics, you can parameterize types, functions, and interfaces, allowing them to operate on different types without explicitly specifying them beforehand.
- Generics allow you to make code more flexible and reusable.

Arrays Definition

```
let fruits: string[] = ['Apple', 'Orange', 'Banana'];
let ids: number[] = [1, 2, 3, 4];

//or using generics
let fruitsG: Array<string> = ['Apple', 'Orange', 'Banana'];
let idsG: Array<number> = [1, 2, 3, 4];

// multi type array
let values: Array<string | number> = ['Apple', 'Orange', 'Banana', 1, 2, 3, 4];

// tuple array
var employee: [number, string][];
employee = [[1, "Steve"], [2, "Bill"], [3, "Jeff"]];

//add element to tuple
var emp: [number, string] = [1, "Steve"];
emp.push(2, 'Bill');
console.log(emp); // [1, "Steve", 2, 'Bill']
```

Union and Intersection Types Examples

```
type MyType = string | number; // Union type
let myUnion: MyType = 'hello'; // Can be a string
myUnion = 123; // Or a number

type TypeA = { name: string };
type TypeB = { age: number };
type CombinedType = TypeA & TypeB; // Intersection
type

let myCombined: CombinedType = { name: 'John', age:
25 }; // Object with both name and age properties
```

Any

Typescript has type-checking and compile-time checks. However, we do not always know the type of some variables, especially when there are user-entered values from third-party libraries. In such cases, we need a provision dealing with dynamic content.

```
let something: any = "Hello world";  
something = 33;  
something = true;  
  
let arr: any[] = ["abc", 34, false];  
arr.push("aaa");  
arr.push(true);
```

Decorators

- Decorators provide a mechanism to add metadata, modify behavior, validate, or extend the functionality of the target element.
- They are functions that execute at runtime.
- Multiple decorators can be applied to a declaration.
- Decorators are experimental features, and the following examples are only compatible with TypeScript version 5 or above using ES6.
- TypeScript versions before 5 should be enabled using the `experimentalDecorators` property in `tsconfig.json` or by using `-experimentalDecorators` in the command line

Decorators

Some of the everyday use cases for decorators include:

- Watching property changes.
- Watching method calls.
- Adding extra properties or methods.
- Runtime validation.
- Automatic serialization and deserialization.
- Logging.
- Authorization and authentication.
- Error guarding.

Note: Decorators for version 5 do not allow decorating parameters.

@Watch Example

```
function Watch(target: any, key: string) {
  let value = target[key];
  const getter = () => value;
  const setter = (newVal: any) => {
    console.log(`Property ${key} changed to: ${newVal}`);
    value = newVal;
  };
  Object.defineProperty(target, key, {
    get: getter,
    set: setter,
  });
}

class Person {
  @Watch
  name: string = "";
}
```

@Serialization Example

```
1  function Serializable(target: any) {
2    | target.prototype.toJSON = function () {
3    |   return JSON.stringify(this);
4    | };
5  }
6
7  @Serializable
8  class Book {
9    | constructor(public title: string, public author: string) {}
10  | }
11
12  const b = new Book("1984", "Orwell");
13  console.log(b.toJSON());
14
```


@Required Example

```
1 function Required(target: any, key: string) {
2   if (!target.__required) target.__required = [];
3   target.__required.push(key);
4 }
5
6 function Validate(instance: any) {
7   const required = instance.__required || [];
8   for (const key of required) {
9     if (!instance[key]) throw new Error(`${key} is required`);
10  }
11 }
12
13 class User {
14   @Required
15   username!: string;
16
17   constructor(data: any) {
18     Object.assign(this, data);
19     Validate(this);
20   }
21 }
22
```

Object Oriented Programming

- TypeScript supports object-oriented class-based programming through the class and interface constructs by which classes and interfaces can be created, respectively.
- As far as the use of classes is concerned, there is no shortage of possibilities for JavaScript to be implemented. Still, TypeScript greatly simplifies this aspect especially about the possibility of making inheritance.

Access modifiers

- Access modifiers in TypeScript
 - `public`: accessible from anywhere (default modifier)
 - `private`: accessible only within the class
 - `protected`: accessible in the class and its subclasses
- Modifiers apply to properties, methods, and constructors

Constructor

- When we define a new class, it automatically has a constructor, the default one.
- We can define a new constructor. When we do, the default one will be deleted.
- When we define a new constructor we can specify each of its parameters with an access modifier and by doing so indirectly define those parameters as instance variables

Constructor - example

- The two following expressions produce the same result

```
class User {  
    private name: string;  
    constructor(name: string)  
    {  
        this.name = name;  
    }  
}
```

```
class User  
{constructor(private name:  
string) {}  
}
```

Instance Variables and Methods

- Variables are usually declared before the constructor.
- Each variable definition includes three parts. The optional access modifier, the identifier and the type annotation.
- Methods are declared without using the function keyword.
- We can precede the function name with an access modifier and add the type of its returned value to the function declaration.

Variables and Static Methods

- It is possible to define static variables and static methods by adding the keyword static.
- Access to static variables and methods is done using the class name.

Example

```
class FinanceUtils {  
    public static VAT = 0.22;  
    public static calculateTax(sum:number):number {  
        return 0.27*sum;  
    }  
  
    public static calculateVAT(sum:number):number {  
        return FinanceUtils.VAT*sum;  
    }  
}  
  
var price:number = 1020;  
document.write("<br/>" + FinanceUtils.calculateVAT(price));
```


Structured Type System

- TypeScript's Structured Type System provides type checking based not only on the name of types, but also on their structure
- The variable of type Person, is initialized with an object of type Student that has within its structure characteristics (properties) also defined in the Person interface (variables defined in the constructor signature of a class with specified visibility, become properties of that class).

Example

- The creation of interfaces and classes in TypeScript is done as follows:

```
interface Person {  
    name: string;  
    surname: string;  
    vat_code?: string;  
}  
  
class Student {  
    private email : string;  
    constructor(public name, public surname) {  
        this.email = name + "." + surname + "@email.it";  
    }  
}  
  
var user: Person = new Student("John", "Doe");
```

After Transpilation

```
var Student = /** @class */ (function () {  
    function Student(name, surname) {  
        this.name = name;  
        this.surname = surname;  
        this.email = name + "." + surname + "@email.it";  
    }  
    return Student;  
})();  
var user = new Student("John", "Doe");
```

Transpilation in Detail

- The transpilation of the OOP constructs provided by TypeScript will produce prototype-based object-oriented JavaScript code, which, since it does not provide for the possibility of distinguishing classes from interfaces, will have no JavaScript counterpart with regard to interfaces, so they will be used exclusively by the compiler to validate the source code.
- It is important to note that the `let` (or `const`) directive must be used for the declaration of a variable, while for the definition of a property of an interface or class, it must not be used.

Abstract methods

- A method can be:
 - Implemented directly in the class
 - Declared without implementation (in abstract classes or interfaces)
- In interfaces, all methods are abstract by definition — the abstract keyword is not required and must not be used
- Example in an abstract class:

```
abstract class Animal {  
    abstract makeSound(): void;  
    move(): void {  
        console.log("Moving...");  
    }  
}
```

Interfaces

- An interface defines the shape of an object.
- It describes the properties and their types, but not implementation.

```
interface Person {  
    name: string;  
    age: number;  
}
```

- Only describes structure
 - No values, no implementation
 - Used mainly for type checking
- Interfaces can be used in different ways

Interface as a type

- Custom object types
- All required properties must be present

```
interface KeyPair {  
    key: number;  
    value: string;  
}
```

```
let kv: KeyPair = { key: 1, value: "Steve" };
```

Interface with optional properties

- Interfaces can have optional properties which can be omitted

```
interface Employee {  
    code: number;  
    name: string;  
    department?: string; // optional  
}
```

```
let e1: Employee = { code: 1, name: "Steve" };  
let e2: Employee = { code: 2, name: "Bill",  
    department: "IT" };
```


Interface with readonly properties

- Interfaces can have readonly properties, their values can not be reassigned after initialization

```
interface Citizen {  
    name: string;  
    readonly ssn: number;  
}
```

```
let c: Citizen = { name: "Anna", ssn: 12345 };  
c.name = "Eva"; // OK  
c.ssn = 99999; // Error: cannot assign to readonly  
property
```

Interface as a function type

- Describes call signatures
- Used to type functions or callbacks

```
interface Processor {  
    (key: number, value: string): void;  
}
```

```
const logKV: Processor = (k, v) =>  
    console.log(`Key=${k}, Value=${v}`);
```

```
logKV(1, "Hello");
```

Interface for array and dictionary types

- Index signatures
- Useful for dynamic keys

```
interface NumberArray {  
    [index: number]: number;  
}
```

```
let arr: NumberArray = [10, 20];
```

```
interface Dictionary {  
    [key: string]: string;  
}
```

```
let d: Dictionary = { TS: "TypeScript", JS: "JavaScript" };
```

Extending Interfaces

- It is possible to extend the interfaces

```
interface IPerson {  
    name: string;  
    gender: string;  
}
```

```
interface IEmployee extends IPerson {  
    empCode: number;  
}
```

```
let empObj:IEmployee = {  
    empCode:1,  
    name:"Bill",  
    gender:"Male"  
}
```

Implementing Interfaces

- How to implement the interfaces

```
interface IEmployee {  
    empCode: number;  
    name: string;  
    getSalary:(empCode: number) => number;  
}  
  
class Employee implements IEmployee {  
    empCode: number;  
    name: string;  
  
    constructor(code: number, name: string) {  
        this.empCode = code;  
        this.name = name;  
    }  
  
    getSalary(empCode:number):number { return 20000; }  
}  
  
let emp = new Employee(1, "Steve");
```

Optional properties and parameters

- Use ? to mark properties or parameters as optional
- Works in interfaces and method definitions
- In classes, a marked property marked with ? exists, but may be undefined
- Behaves like overloading for simple cases but doesn't cover different parameter types — use overloads instead

```
interface User {  
    name: string;  
    age?: number;  
}
```

```
function greet(name?: string): void {  
    console.log("Hello", name ?? "guest");  
}
```

Function Overloading

- TypeScript allows multiple function signatures (overloads)
- These signatures are only checked at compile time
- Overloads merely describe how the function can be called
- One actual implementation must be provided
 - This is the only version that gets compiled to JavaScript
 - It must handle all overload cases
- The implementation must be compatible with all overloads

Function Overloading - example

```
// Overload signatures (for the compiler)
function combine(a: number, b: number): number;
function combine(a: string, b: string): string;

// Implementation (required!)
function combine(a: number | string, b: number |
string): number | string {
    return typeof a === "number" && typeof b === "number"
        ? a + b
        : a.toString() + b.toString();
}
```


Polymorphism

- Polymorphism allows writing code that works with different types through a shared interface
- In classic OOP, this is done through inheritance and interface implementation
- TypeScript supports this, but also allows structural polymorphism
 - Based on shape compatibility, not explicit inheritance
 - Works for different types as long as they structurally match
- Union types are a powerful way to express structural polymorphism

Inheritance Polymorphism - Example

```
class Animal {  
    speak(): void {  
        console.log("Arf!");  
    }  
}  
  
class Dog extends Animal {  
    speak(): void {  
        console.log("Woof!");  
    }  
}
```

```
class Pig extends Animal {  
    speak(): void {  
        console.log("Breee!");  
    }  
}  
  
function makeItSpeak(a: Animal): void {  
    a.speak();  
}  
  
makeItSpeak(new Dog()); // Woof!  
makeItSpeak(new Pig());  
makeItSpeak(new Pig());  
makeItSpeak(new Pig());  
// Breee! Breee! Breee!
```

Structural Polymorphism - Example

```
type Book = {  
  title: string;  
  author: string;  
};  
  
type Movie = {  
  title: string;  
  director: string;  
};  
//Union type  
type Media = Book | Movie;  
//Polymorph function  
function printLabel(item: Media): void {  
  console.log(`Title: ${item.title}`);  
}
```

Modules

- TypeScript allows the creation of modules, a useful concept for structuring source code and taking advantage of reusability.
- A module has a name and a set of TypeScript concepts (classes, interfaces, variables, functions, etc.);
- Anything you want to make usable of a module by the component that uses it should be declared with the keyword `export` at the beginning of the definition of the construct itself.

Example

```
module MathModule {  
    export class Expression {  
        sum(term1: number, term2: number) {  
            return term1 + term2;  
        }  
    }  
}  
  
var expr = new MathModule.Expression();  
var res = expr.sum(1, 2);
```

Usage of Javascript libraries

- An important feature of TypeScript is being compatible with all existing libraries for JavaScript
- A library can be used in a TypeScript source without any addition of further code, but a file with the extension `.d.ts` must necessarily be created containing the definition of the data types and interfaces used by that library
- In some cases, where there are libraries that provide utilities through a single global object, the `d.ts` file will also contain the declaration of such a variable that can be used within the TypeScript application to then refer to the library itself

TypeScript Code Generation

- The TypeScript transpiler has two primary responsibilities: checking for type errors and transpiling to JavaScript.
- These two processes are independent of each other.
- Types do not affect the execution of the code in a JavaScript runtime, as they are entirely erased during compilation.
- TypeScript can still output JavaScript even in the presence of type errors.

TypeScript Type Error

- Here is an example of TypeScript code with a type error:

```
const add = (a: number, b: number): number => a + b;  
const result = add('x', 'y'); // Argument of type  
'string' is not assignable to parameter of type  
'number'.
```

- However, it can still produce executable JavaScript output:

```
'use strict';  
const add = (a, b) => a + b;  
const result = add('x', 'y'); // xy
```


TypeScript Type Error

- It is not possible to check TypeScript types at runtime.

```
1 interface Animal {
2   name: string;
3 }
4 interface Dog extends Animal {
5   bark: () => void;
6 }
7 interface Cat extends Animal {
8   meow: () => void;
9 }
10 const makeNoise = (animal: Animal) => {
11   if (animal instanceof Dog) {
12     // 'Dog' only refers to a type, but is being used as a value here.
13   }
14 };
15
```

- As the types are erased after compilation, there is no way to run this code in JavaScript.

```
11 | if (animal instanceof Dog) {
    |                                     ^
  typescript.ts:1 of 1 problem
'Dog' only refers to a type, but is being used as a value here. ts(2693)
```

Recognizing Types at Runtime

- To recognize types at runtime, we need to use another mechanism.
- TypeScript provides several options, a common one being "tagged union."
- The property "kind" is a value that can be used at runtime to distinguish between objects in JavaScript.
- It is also possible for a value at runtime to have a type different from the one declared in the type declaration.
- For instance, if the developer has misinterpreted an API type and annotated it incorrectly.

```
1 interface Dog {
2     kind: 'dog'; // Tagged union
3     bark: () => void;
4 }
5
6 interface Cat {
7     kind: 'cat'; // Tagged union
8     meow: () => void;
9 }
10
11 type Animal = Dog | Cat;
12
13 const makeNoise = (animal: Animal) => {
14     if (animal.kind === 'dog') {
15         animal.bark();
16     } else {
17         animal.meow();
18     }
19 };
20
21 const dog: Dog = {
22     kind: 'dog',
23     bark: () => console.log('bark'),
24 };
25
26 makeNoise(dog);
```

Recognizing Types at Runtime

- TypeScript is a superset of JavaScript, so the "class" keyword can be used as a type and value at runtime.
- In JavaScript, a "class" has a "prototype" property, and the "instanceof" operator can be used to test if the prototype property of a constructor appears anywhere in the prototype chain of an object.
- TypeScript does not affect runtime performance, as all types will be erased.
- However, TypeScript does introduce some build time overhead.

```
1  class Animal {
2      constructor(public name: string) {}
3  }
4  class Dog extends Animal {
5      constructor(
6          public name: string,
7          public bark: () => void
8      ) {}
9      super(name);
10 }
11
12 class Cat extends Animal {
13     constructor(
14         public name: string,
15         public meow: () => void
16     ) {}
17     super(name);
18 }
19
20
21 type Mammal = Dog | Cat;
22
23 const makeNoise = (mammal: Mammal) => {
24     if (mammal instanceof Dog) {
25         mammal.bark();
26     } else {
27         mammal.meow();
28     }
29 };
30
31 const dog = new Dog('Fido', () => console.log('bark'));
32 makeNoise(dog);
33
```

Promise

- A Promise represents the eventual result of an asynchronous operation.
- It can be in one of three states:
 - pending – initial state
 - fulfilled – operation completed successfully
 - rejected – operation failed

```
const promise: Promise<string> = new Promise((resolve, reject) => {  
    setTimeout(() => resolve("done!"), 1000);  
});  
promise.then(result => console.log(result)); // prints "done!"  
after 1s
```

- Promises are the foundation of async/await
- In TypeScript, always annotate the resolved type with Promise<T>

async/await

- async/await provides typed, readable asynchronous code.
- TypeScript infers return types from async functions.
- Built on top of Promises – fully compatible with type checking.

```
async function fetchUser(): Promise<string> {  
    return "user123";  
}
```

- The function returns a `Promise<string>`
- Type is inferred, but can be explicitly annotated

Using await to Handle Promises

- Awaited expressions **can be typed**
- Compiler ensures correct type usage

```
async function getUser(): Promise<void> {  
    const response: Response = await fetch("/api/user");  
    const data: { name: string } = await response.json();  
    console.log(data.name);  
}
```

Type Inference in Async Functions

- Since return type can be automatically inferred, the two following examples are equivalent

```
async function getAnswerToEverything (): Promise<number> {  
    return 42;  
}
```

```
async function getAnswerToEverything () {  
    return 42;  
}
```

42

- TypeScript infers the return type as `Promise<number>`
- Explicit typing is useful when:
 - The return type is complex
 - You want to enforce a specific return contract

Handling Promises

- Native promise syntax (`.then()` chaining)

```
fetch("/data")  
  .then(res => res.json())  
  .then(data => console.log(data));
```

- `async/await` syntax

```
const res = await fetch("/data");  
const data = await res.json();
```

- TypeScript supports both, but `async/await` syntax is cleaner and easier to type-check

Parallelization of Async Tasks

- Use `Promise.all()` to run multiple async tasks in parallel
- Tuple typing ensures expected types for multiple results

```
async function loadAll(): Promise<void> {  
    const [user, posts]: [User, Post[]] = await Promise.all([  
        fetchUser(),  
        fetchPosts()  
    ]);  
}
```

- Both requests are started at the same time
- Results are returned together in a tuple

Parallelization of Async Tasks: iterations

- Avoid using `await` in iterations (e.g. `forEach()` or `map()`) without `Promise.all()`
 - The following code runs one request at a time
 - Total time = sum of all durations

```
for (const id of ids) {  
  await fetchUser(id);  
}
```

- The following code starts all the request at the same time
- Total time = duration of the slowest request

```
await Promise.all(ids.map(id => fetchUser(id)));
```

Error Handling

- TypeScript uses the same try/catch mechanism as Java and JavaScript
- You can catch errors from:
 - throw statements
 - rejected Promises (with `await` or `.catch()`)
 - unexpected runtime failure (e.g. accessing a property/method on an undefined or null object)
 - ...
- TypeScript adds type safety to error handling:
 - The error object caught in a catch block has type `unknown`
 - This prevents unsafe access to its properties
- You must narrow the type before using it (e.g. with `instanceof`)

Error Handling with `try/catch`

- TypeScript forces you to treat `err` as unknown
- This prevents runtime crashes when `err` is not what you expect

```
async function loadData(): Promise<void> {  
  try {  
    await fetchData(); // might throw or reject  
  } catch (err) {  
    console.error(err.message); //Error: Property  
    'message' does not exist on type 'unknown'  
  }  
}
```

Safe error handling with `instanceof`

- `instanceof Error` is the most common type guard
- Ensures that `err` has a `.message`, `.stack`, etc.

```
async function loadData(): Promise<void> {  
  try {  
    await fetchData();  
  } catch (err) {  
    if (err instanceof Error) {  
      console.error(err.message); //Safe access  
    } else {  
      console.error("Unexpected error", err);  
    }  
  }  
}
```

Type guard with `hasOwnProperty()`

- `hasOwnProperty()` lets you check if a property exists safely
- Works even if the object doesn't inherit from `Object.prototype`
- Safer than `err.message !== undefined` in loosely-typed environments

```
try {  
  throw { message: "API limit exceeded", code: 429 };  
} catch (err) {  
  if (  
    typeof err === "object" && err !== null &&  
    Object.prototype.hasOwnProperty.call(err, "message")  
  ) {  
    console.error("Handled error:", (err as { message: string }).message);  
  } else {  
    console.error("Unknown error", err);  
  }  
}
```

Throwing errors

- Always throw objects, not strings or numbers
- Prefer Error, or custom subclasses like NotFoundError, BadRequestError, etc.
- You can define your own error classes for domain-specific logic:

```
class NotFoundError extends Error {  
    constructor(message: string, public status: number) {  
        super(message);  
    }  
}  
  
throw new NotFoundError(404, "Resource not found");
```

- Avoid throwing:
 - throw "oops" (string)
 - throw 123 (number)
 - throw undefined